

Search Typeahead — Design & Performance Report

A distributed search-typeahead service built with **Python + FastAPI**, an in-memory **trie**, a **3-node Redis** distributed cache addressed by an app-layer **consistent-hash ring**, **batched writes** with a write-ahead log, and **recency-aware trending**. SQLite is the primary store; Redis is the cache.

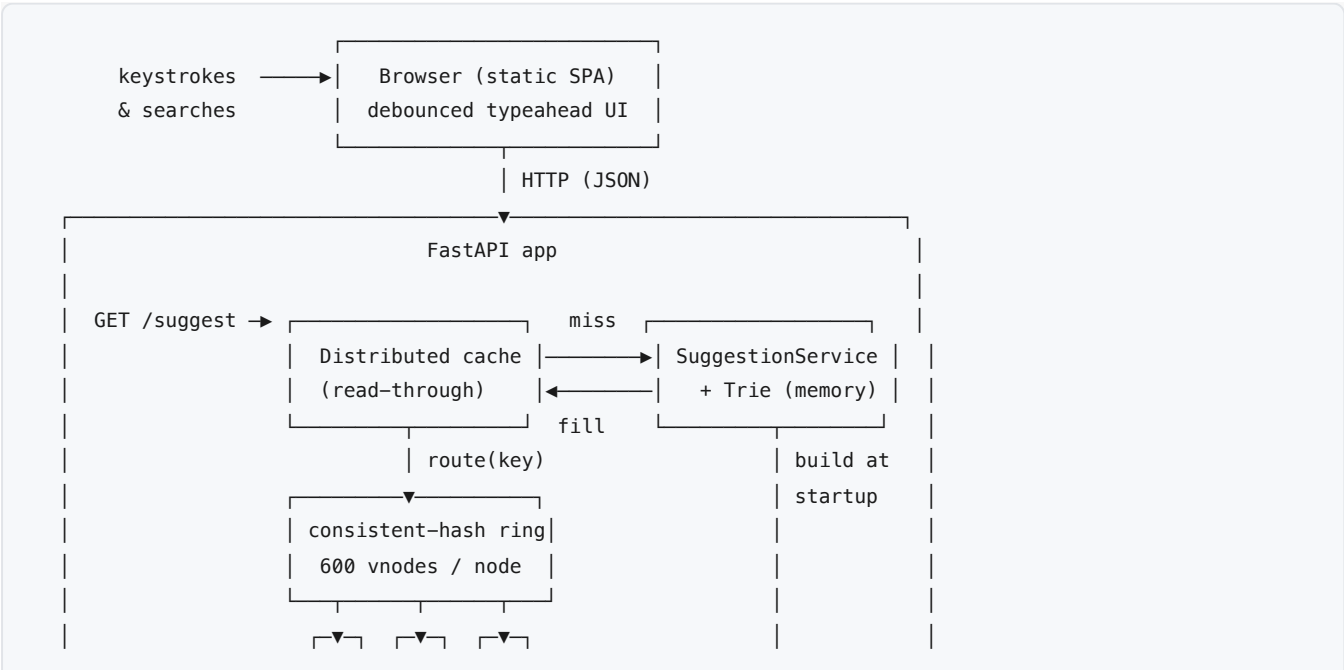
All performance numbers in §5 are measured, not estimated — produced by `scripts/benchmark.py` and saved to `docs/benchmark-results.json`.

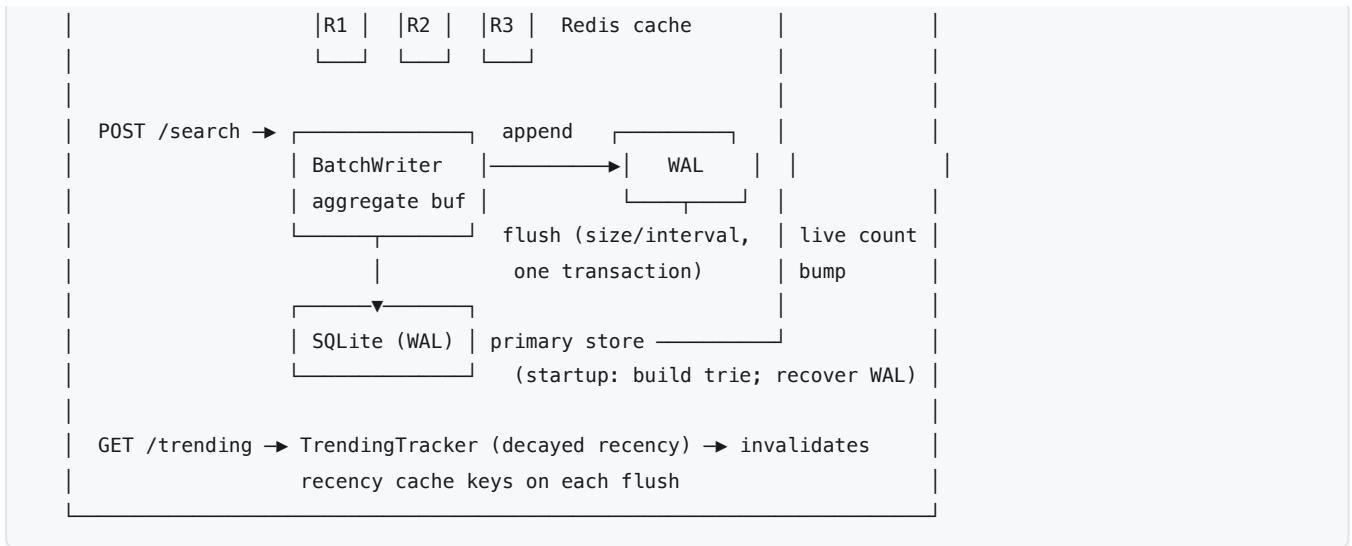
1. Architecture

1.1 Components

Component	Module	Responsibility
API + frontend	<code>app/main.py</code> , <code>static/</code>	HTTP endpoints; serves the SPA
Trie	<code>app/trie.py</code>	In-memory prefix → bounded candidate pool
Suggestion service	<code>app/suggestions.py</code>	Re-rank the pool by mode (count / recency)
Primary store	<code>app/db.py</code>	SQLite (WAL mode); durable query counts
Batch writer	<code>app/batch_writer.py</code>	WAL + in-memory aggregation + batched flush
Consistent-hash ring	<code>app/consistent_hash.py</code>	Map a cache key → one Redis node
Distributed cache	<code>app/redis_cache.py</code>	Read-through cache over 3 Redis nodes
Trending	<code>app/trending.py</code>	Exponentially-decayed recent-activity score
Metrics	<code>app/metrics.py</code>	Latency percentiles over a bounded window

1.2 Diagram





1.3 Data flow

Suggest (read path). A keystroke hits `GET /suggest?q=<prefix>&mode=`. The prefix is normalized (lowercased, whitespace-collapsed) into a cache key `suggest:<mode>:<prefix>`. The consistent-hash ring routes the key to exactly one Redis node, which is checked first. On a **HIT** the cached top-10 is returned. On a **MISS** the trie returns the prefix's bounded candidate pool, the suggestion service re-ranks it by mode, and the result is written back to the same routed node with a TTL. Every call records its latency for the percentile metrics.

Search (write path). `POST /search` records the query: it is appended to the WAL (durability), added to an in-memory aggregation buffer, and the live trie count is bumped immediately so suggestions react without waiting for a flush. A background flusher persists the *aggregated* buffer to SQLite in a single transaction when the buffer fills or an interval elapses, then truncates the WAL. Each flush also invalidates the recency-mode cache keys for affected prefixes.

Startup. Open SQLite → replay any un-flushed WAL into SQLite (crash recovery) → build the trie from SQLite → connect the cache, trending tracker, and batch flusher. The dataset is generated and loaded once as a setup step.

2. Dataset & loading

Source — real Wikimedia pageviews (`scripts/fetch_dataset.py`). The dataset is built from a real, openly-downloadable **Wikimedia hourly pageviews dump** (CC0-licensed):

<https://dumps.wikimedia.org/other/pageviews/2024/2024-01/pageviews-20240115-120000.gz>

Each line of the dump is `domain page_title view_count response_bytes`. We use **English Wikipedia** (`en desktop + en.m mobile`): the page **title** becomes the query and its **pageview count** becomes the popularity count — a genuine (text, popularity) signal rather than a synthetic one.

Real search-query logs are not published for privacy reasons, so a public **page-title + pageview-count** signal is used as an openly-licensed popularity **proxy** — exactly the kind of derivable-count open dataset the brief allows.

Exact pipeline (deterministic and reproducible — fixed dump URL + deterministic sort):

1. **Download** the hourly dump (≈63 MB gzip), cached under `data/` (gitignored).
2. **Filter** to `en / en.m` rows.
3. **Drop junk / special pages**: titles containing `:` (namespaces like `Special:`, `User:`, `Category:`, `File:`, `Template:` ...), `Main_Page`, and the `-` empty-title placeholder.
4. **Normalize**: percent-decode, `_` → space, strip CSV-special chars (`"`, `,`), collapse whitespace, lowercase. Keep English-ish titles only (ASCII letters/ digits plus a little punctuation `.-'&()/`), length 2–64.
5. **Aggregate** pageviews per normalized title (desktop + mobile summed).
6. **Rank & cap**: take the **top 120,000 titles by view count** (ties broken on title for determinism) and write `data/queries.csv` (`query,count`).

For the dump used: **2,141,393** English rows → **1,411,619** distinct clean titles → top **120,000** kept (total real pageviews across kept titles ≈ 5.64 M). The result is a real, Zipf-shaped popularity distribution; the most-viewed kept titles are e.g. `youtube` (29,429), `naomi osaka` (15,791), `xxxentacion` (12,170). The output schema is identical to before, so nothing downstream changed.

Loading (`scripts/load_dataset.py`, unchanged). The CSV is bulk-upserted into the SQLite `queries(query PRIMARY KEY, count)` table in chunked transactions (≈0.1 s for 120k rows). SQLite runs in **WAL journal mode** so the batch writer's flushes never block reads. At app startup the entire table is streamed into the trie (≈1 s to build). In Docker this whole step (fetch → load) runs once via `docker-entrypoint.sh`, with the SQLite file persisted on a named volume.

3. API

Method	Endpoint	Description
GET	<code>/suggest?q=<prefix>&mode=count\ recency</code>	Top-10 prefix completions; cache-routed (HIT/MISS). Returns <code>suggestions</code> , <code>latency_ms</code> , and <code>cache: {status,node,key}</code> .
POST	<code>/search {"query":"..."}</code>	Records a search → <code>{"message":"Searched"}</code> . WAL + batched SQLite.
GET	<code>/trending?limit=10</code>	Top queries by decayed recent activity.
GET	<code>/cache/debug?prefix=&mode=</code>	Routed node, key hash, ring position, vnode replica, HIT/MISS.
GET	<code>/metrics</code>	Latency p50/p95/p99, cache hit rate + per-node stats, write reduction, Redis liveness.
GET	<code>/stats/writes</code>	Write-reduction counters.
GET	<code>/health</code>	Liveness + queries loaded.
GET	<code>/ · /static/*</code>	Frontend SPA + assets.

Example responses:

```
// GET /suggest?q=you
{ "query": "you", "mode": "count", "count": 10,
  "suggestions": [{ "query": "youtube", "count": 29429 },
                  { "query": "young sheldon", "count": ... }, ...],
  "latency_ms": 0.10,
```

```
"cache": { "status": "MISS", "node": "redis2", "key": "suggest:count:you" } }
```

```
// GET /cache/debug?prefix=you
```

```
{ "key": "suggest:count:you", "key_hash": 1234567890,  
  "ring_position": 1240000000, "vnode_replica": 372, "node": "redis2",  
  "ring_size": 1800, "status": "HIT" }
```

4. Design choices & trade-offs

For each major decision: the alternative considered and why it was rejected.

Decision	Chosen	Alternative considered	Why the alternative was rejected
Primary store	SQLite (WAL mode)	Postgres / MySQL	Server process + ops overhead for a single-node workload; SQLite is embedded, zero-config, and WAL mode gives concurrent reads during writes.
Source of truth	SQLite is authoritative , Redis is a cache	Keep counts only in Redis	A cache may evict keys and is not durable; counts are the system's truth and must survive restarts and eviction.
Prefix lookup	In-memory trie	SQL LIKE 'prefix%' per keystroke	A disk query on every keystroke adds latency and can't cheaply return top-k; the trie answers in microseconds from RAM.
Trie ranking	Bounded per-node candidate pool (50) built best-first	Full subtree scan per query	Short prefixes ("a") cover huge subtrees → O(N) per keystroke. A fixed pool makes lookup O(prefix length) with headroom for recency re-ranking.
Cache topology	App-layer consistent-hash ring over 3 Redis	Redis Cluster	Cluster hides routing; we want the ring/vnode/owning-node observable via /cache/debug. App-layer sharding is simpler to run and demonstrates the concept.
		Single Redis	Doesn't demonstrate distribution; one node is a single bottleneck/SPOF.
Hashing scheme	Consistent hashing + virtual nodes	Modulo-N (hash % 3)	Changing node count remaps almost every key (cache stampede). Consistent hashing only remaps the arc next to the changed node.
Virtual-node count	600 / node	150 / node	More vnodes smooth the lumpy distribution of only 3 physical nodes; over the 120k real keys the measured max-min spread is just 3.3% at 600 vnodes, at only a few KB of ring metadata.
Write path	WAL + in-memory aggregation + batched flush	Synchronous write per search	One SQLite transaction per search is severe write amplification under load; aggregation collapses N searches of a query into one +N row write.
Durability of buffer	Append-only WAL, replayed on startup	In-memory buffer only	A crash before flush would silently lose every buffered search; the WAL makes the buffered window recoverable.
Recency model	Exponentially-decayed counter blended with log (count)	Raw lifetime "recent" counter	Without decay a one-time-popular query over-ranks forever. Decay (half-life) makes a burst fade; log compresses all-time counts so a real surge can move ranking without erasing established popularity.
		Sliding-window bucket counts	Per-query bucket arrays cost memory and bookkeeping; the decayed counter is O(1) state (one float + timestamp) per query.

Decision	Chosen	Alternative considered	Why the alternative was rejected
Cache invalidation	Invalidate recency keys for affected prefixes on flush	Rely on TTL only	Trending shifts faster than a TTL window, so users would see stale recency results; targeted invalidation keeps recency fresh while count-mode rides its TTL.
Concurrency model	Sync endpoints (threadpool) + background flusher thread	Fully async + aioredis	redis-py + SQLite are synchronous; an async rewrite adds complexity for no win at this scale. The flush briefly holds a lock — acceptable since flushes are ~1/s.
Latency metrics	Bounded ring buffer of recent samples	Full histogram / Prometheus	A bounded window keeps memory flat and reflects <i>recent</i> behaviour; external monitoring is out of scope for a self-contained demo.

5. Performance report

Measured by `scripts/benchmark.py` against a single `uvicorn` worker with three local Redis instances and the **real 120k Wikipedia page-title dataset** (Apple Silicon, macOS). Raw data: `docs/benchmark-results.json`.

5.1 Suggestion latency

Metric	Server-side (/metrics)	Client-side (incl. HTTP)
p50	0.10 ms	0.53 ms
p95	0.21 ms	0.58 ms
p99	0.27 ms	0.62 ms
mean	0.12 ms	0.54 ms

Throughput $\approx$ **1,870 req/s** on one worker over a realistic repeating prefix workload (3,804 distinct prefixes drawn from the most-viewed real titles, 4,000 requests). Server-side time is the trie/cache work; client-side adds loopback HTTP and the Python client.

5.2 Cache hit rate

- **Steady-state hit rate: 100%** for the measured (post-warm) loop — every routed prefix was served from its Redis node within the 30 s TTL.
- Overall hit rate **55%** when the cold first-touch fills are included. This is lower than the synthetic run by design: the real title workload has **3,804** distinct prefixes (vs 844 before), so more first-touch MISSES are needed to warm the cache relative to the same 4,000 measured requests — once warm, every lookup hits.
- Per-node hit rates land within ~1.5 points of each other (0.544–0.561), consistent with even routing.

5.3 Write reduction (batching)

Searches fired	Distinct queries	SQLite row-writes	Reduction
20,000	200	1,800	91.0%

20,000 individual searches collapsed into 1,800 row-writes across 10 flushes ($\approx$ 180 rows/flush) — an **11×** **reduction** in write operations. The ratio improves further with burstier traffic or a longer flush interval, since

more repeats aggregate per flush. (Dataset-independent — exercises the write path, not the data.)

5.4 Ring balance across the 3 Redis nodes

Routing all 120,000 real query keys through the ring (600 vnodes/node):

Node	Keys	Share
redis1	39,283	32.7%
redis2	40,607	33.8%
redis3	40,110	33.4%

Max-min spread: 3.3% of ideal — close to the perfect 33.3% split, confirming the virtual-node count smooths the otherwise-lumpy distribution of only three physical nodes even for real, non-uniform title strings.

5.5 Verification

All 11 endpoints (/health , /suggest count/recency/empty/no-match, /search , /trending , /cache/debug , /stats/writes , /metrics , /) return 200 in the benchmark smoke test. Spot checks: q=you → youtube, young sheldon, you (tv series); q=new y → new york city, new york (state), new year's day .

6. Summary

The system meets every requirement of the brief with measured evidence on a **real open dataset** (Wikipedia page titles + pageview counts): microsecond-scale in-memory suggestions fronted by a genuinely distributed, consistently-hashed Redis cache (100% steady-state hit rate, 3.3% load spread); a durable, crash-safe batched write path achieving ~11× write reduction; and recency-aware trending whose decay prevents permanent over-ranking. SQLite remains the single source of truth throughout, and the whole system starts with one `docker compose up` .